DWH MCP Explorer

Этап 6.9.1 | Итоговая архитектура системы | Базовая версия архитектуры v1.0

Статус: документ основан на реализованной функциональности этапов 5–6 и результатах тестирования по состоянию на 7 ноября 2026 года. Это срез архитектуры, а не готовый к публичному развёртыванию репозиторий.

1. Назначение и область применения

DWH MCP Explorer — собственный MCP-сервер на Python, реализующий Model Context Protocol. Он позволяет AI-клиенту Claude Desktop обнаруживать и интерпретировать метаданные хранилища данных, определения SQL и зависимости данных (lineage), сравнивать маппинги и проверять отдельные правила качества данных. В одной системе сосуществуют два направления: локальный PostgreSQL с репозиторием SQL-файлов и Azure Databricks Lakehouse.

2. Высокоуровневая архитектура

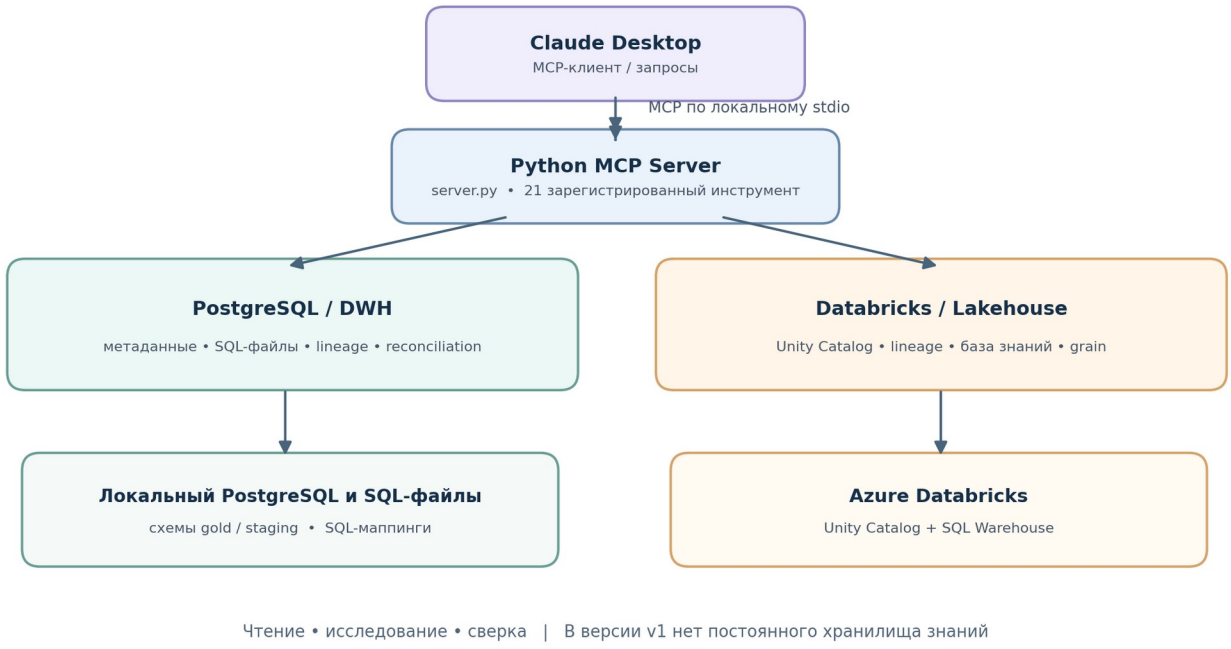


Рисунок 1. Логическая архитектура и основные потоки данных.

3. Основные компоненты приложения

Уровень / модуль	Назначение
Claude Desktop	MCP-клиент: обнаруживает инструменты, вызывает операции и формирует технические отчёты.
server.py	Тонкий адаптер MCP и реестр инструментов; предоставляет клиенту 21 инструмент.
dwh_mcp/database.py, sql_repository.py	Доступ к PostgreSQL, обнаружение SQL-файлов и чтение их содержимого.
dwh_mcp/lineage.py, reconciliation.py, reporting.py, composition.py	Извлечение lineage, сверка структуры и ограничений, формирование результатов и отчётов.
dwh_mcp/databricks_provider.py	Исследование Unity Catalog: каталоги, схемы, таблицы,

	метаданные и колонки.
dwh_mcp/databricks_lineage_provider.py	Запросы к system.access.table_lineage и system.access.column_lineage через Databricks SQL Statement Execution API.
dwh_mcp/databricks_knowledge.py	Объединение метаданных и lineage таблиц/колонок в единый объект знаний, создаваемый по запросу в режиме только чтения.
dwh_mcp/databricks_reconciliation.py, databricks_grain_provider.py	Безопасное построение SQL и точные проверки зерна (grain) / бизнес-ключа; преобразование SQL-результатов в отчёт о валидации.

4. Перечень инструментов MCP (21)

4.1 PostgreSQL / SQL / DWH (13 инструментов)

list_schemas; list_tables; describe_table; get_primary_keys; get_foreign_keys; list_sql_files; read_sql_file; get_table_knowledge; get_table_lineage; reconcile_table_mapping; reconcile_column_constraints; generate_reconciliation_report; investigate_dwh_object.

4.2 Databricks (8 инструментов)

databricks_list_catalogs; databricks_list_schemas; databricks_list_tables; databricks_describe_table; databricks_get_table_lineage; databricks_get_column_lineage; databricks_investigate_object; databricks_validate_grain.

5. Основные сценарии обработки запросов

Сценарий	Последовательность / результат
Исследование метаданных	Claude → MCP Tool → Unity Catalog API либо PostgreSQL metadata → структурированный ответ → объяснение AI.
Объект знаний Databricks	Claude → databricks_investigate_object → провайдер метаданных + lineage таблиц + lineage колонок → единый JSON (метаданные, зависимости, источники подтверждения, ограничения).
Проверка grain	Claude → databricks_validate_grain(full_name, key_column) → проверка идентификаторов → агрегирующий SQL только для чтения через SQL Warehouse → число строк, уникальных ключей, NULL, избыточных дубликатов, статус passed/failed.
Сверка SQL и lineage	Claude → инструменты PostgreSQL/SQL → метаданные + SQL-маппинг / lineage → сравнение и отчёт; набор правил ограничен реализованными проверками.

6. Границы доверия, эксплуатационные меры и ограничения

- Аутентификация: Databricks SDK использует именованный профиль CLI/OAuth. Учётные данные и refresh-токены находятся вне исходного кода; локальная конфигурация Claude запускает Python MCP-сервер.
- Безопасность SQL: проверка grain принимает простое трёхчастное имя таблицы и одно имя ключевой колонки, валидирует их, экранирует идентификаторы и выполняет SELECT в режиме только чтения. Трансформации SQL не выполняются.
- Полнота lineage: история запросов ограничена (сейчас окно 45 дней и 50 событий на запрос). Отсутствие признака усечения в ответе SDK не доказывает полноту графа зависимостей.
- Интерпретация: события чтения не эквивалентны подтверждённым связям источник → приёмник; column lineage не содержит выражения преобразований.
- Knowledge Base v1: JSON-объект формируется по запросу, отдельного постоянного индексированного хранилища знаний нет. Поле execution_context сейчас пусто; идентификаторы исполнителей могут присутствовать в событиях lineage.
- Качество данных: статистика Unity Catalog может быть приблизительной. Для точного подтверждения уникальности и NULL необходимо выполнить агрегирующий SQL. Проверка gold.fact_orders дала 10 000 строк, 10 000 уникальных order_id, 0 NULL и 0 избыточных дубликатов.

- Эксплуатация: запросы к Databricks SQL Warehouse могут создавать расходы; срок действия OAuth-учётных данных ограничен, поэтому иногда требуется повторная авторизация.

7. Доказательства проверок и текущий статус

Группа проверок	Подтверждение / результат
Stage 6.6 — Knowledge Object	Метаданные и lineage таблиц/колонок объединены; подтверждены реальный тест в Databricks и сквозная проверка через Claude Desktop.
Stage 6.7 — Grain Validation	Пройдены Mock-тесты провайдера, реальный SQL-тест Warehouse, вызов MCP-клиента и сквозная проверка Claude Desktop.
Stage 6.8 — регрессия	7/7 актуальных тестов; реестр двух платформ; негативные сценарии; флаг ошибки MCP; 5/5 регрессионных тестов прежних модулей.
Зарегистрированные инструменты	MCP-клиент видит 21 инструмент; группы PostgreSQL/DWH и Databricks зарегистрированы совместно.

8. Что не входит в текущую реализацию и дальнейшие шаги

В текущей архитектуре не реализованы: постоянный поиск/индекс по объектам знаний, полный обход графа lineage, автоматическое преобразование ID заданий и notebooks в имена и восстановление SQL-выражений, а также полная межплатформенная сверка всех семантических и бизнес-правил. Этапы 6.9.2–6.9.5 детализируют контракты инструментов, подтверждения тестирования, развертывание/конфигурацию и закрытие проекта. Перед публикацией на GitHub необходимо отдельно проверить конфигурацию на наличие секретов.